

Python Functions: for beginner

Python functions are one of the most important building blocks in programming. They help you organize code, reuse logic, and make programs easier to understand. Let's walk through the basics in a simple way.

◆ What is a Function?

A function is a block of code that runs when you call it.

Instead of writing the same code again and again, you put it inside a function and reuse it.

◆ Creating a Function

In Python, you use the keyword `def` :

</> Python



▶ Run

```
def greet():  
    print("Hello, world!")
```

◆ Calling a Function

To run the function:

</> Python



▶ Run

```
greet()
```

Output:

Hello, world!



◆ Function with Parameters

You can pass data into a function:

</> Python



▶ Run

```
def greet(name):  
    print("Hello,", name)
```

Call it like this:

</> Python



▶ Run

```
greet("Arun")
```

Output:

Hello, Arun



◆ Function with Return Value

Functions can return results using `return` :

</> Python



▶ Run

```
def add(a, b):  
    return a + b
```

Use it like:

</> Python



▶ Run

```
result = add(5, 3)  
print(result)
```

Output:

8



◆ Default Parameters

You can give default values:

</> Python



▶ Run

```
def greet(name="Guest"):
    print("Hello,", name)
```

</> Python



▶ Run

```
greet()          # Hello, Guest
greet("Ravi")    # Hello, Ravi
```

◆ Keyword Arguments

You can pass arguments by name:

</> Python



▶ Run

```
def student(name, age):
    print(name, age)

student(age=20, name="Priya")
```

◆ Why Use Functions?

- Avoid repeating code
- Make programs cleaner
- Easier to debug
- Reusable in different parts of program

◆ Simple Practice Examples

1. Even or Odd

</> Python



▶ Run

```
def check_even(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

2. Square of a Number

</> Python



▶ Run

```
def square(n):  
    return n * n
```

◆ Quick Tip

- Function names should be **clear and meaningful**
- Use lowercase with underscores: `my_function_name`

If you want, I can give you **practice problems**, a **mini project**, or explain **advanced topics** like **lambda**, **recursion**, **decorators** step by step.



why def keyword?

Good question—this gets to the core of how Python is designed.

◆ Why def Keyword?

The `def` keyword in Python stands for “**define**”. It tells Python:

👉 “I am creating (defining) a function here.”

◆ What Happens When You Use `def` ?

When Python sees this:

</> Python



▶ Run

```
def greet():  
    print("Hello")
```

It does **not run the code immediately**. Instead, it:

1. Creates a function object
2. Stores it in memory
3. Gives it the name `greet`

The function only runs when you call it:

</> Python

```
greet()
```



▶ Run

◆ Why Not Just Write Code Directly?

You *can* write code directly, but `def` helps you:

✓ Reuse code

</> Python

```
def add(a, b):  
    return a + b
```



▶ Run

Instead of rewriting `a + b` many times.

✓ Organize programs

Functions break big programs into smaller parts.

✓ Improve readability

</> Python

```
calculate_total()
```



▶ Run

is much clearer than repeating many lines of code.

◆ Why Python Uses the Word `def` ?

Python is designed to be **simple and readable**.

Instead of symbols or complex syntax, it uses clear words:

- `def` → define a function
- `if` → condition
- `for` → loop

So `def` is short, readable, and easy to remember.

◆ Bonus: Function Without `def` ?

Yes—Python also has **lambda functions** (short, one-line functions):

⌞ Python

```
add = lambda a, b: a + b
```



▶ Run

But these are limited. For most cases, `def` is better.

◆ Simple Analogy

Think of `def` like saving a recipe:

- Writing recipe → `def`
 - Cooking food → calling the function
-

If you want, I can ↪ explain how Python stores functions internally or ↪ show visual step-by-step execution.

